

4주차 예습과제

Ⓢ 과제 종류	예습과제
≡ 범위	딥러닝 파이토치 5장

Chapter 05. 합성곱 신경망 I - Part2

5.3 전이학습

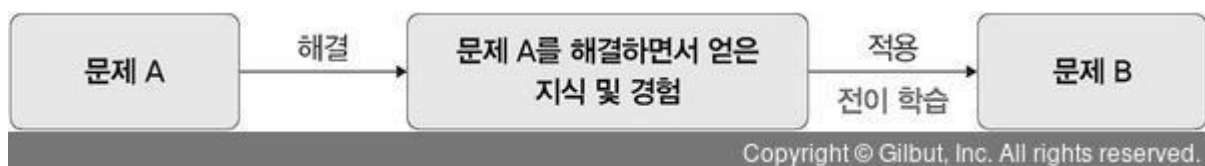
- 합성곱 신경망 기반의 딥러닝 모델을 제대로 훈련시키려면 많은 양의 데이터가 필요하지만 돈/시간을 이유로 큰 데이터셋을 확보하기 어려움.
- 해결 방법 : **전이 학습**(Transfer learning)

전이학습

'이미지넷처럼 아주 큰 데이터셋을 써서 훈련된 모델의 가중치'를 가져와, 해결하려는 과제에 맞게 보정해서 사용하는 것

+) 아주 큰 데이터셋을 사용해 훈련된 모델 = **사전 훈련된 모델(네트워크)**

→ 전이학습을 사용하면 '비교적 적은 수의 데이터'로, 우리가 원하는 과제 해결 가능



- 전이학습을 위한 방법: **1. 특성 추출, 2. 미세 조정**

5.3.1 특성 추출 기법

특성 추출 과정

- 이미지넷 데이터셋으로 **사전 훈련된 모델**을 가져옴

2. 마지막에 완전연결층 부분만 새로 만들

→ 학습할때 마지막 완전연결층(이미지의 카테고리를 결정하는 부분)만 학습, 나머지 계층들은 학습X

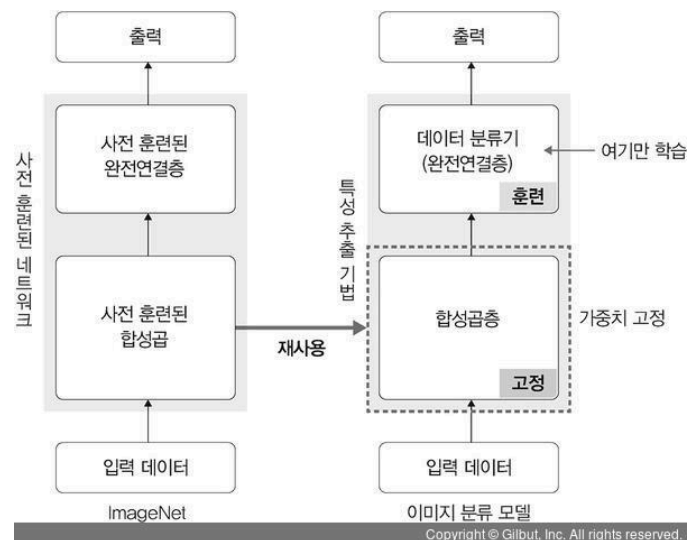
특성 추출 구성

- 합성곱층 : 합성곱층 + 풀링층
- 데이터 분류기(완전 연결층) : 추출된 특성을 입력받아, 최종적으로 이미지에 대한 클래스를 분류하는 부분

→ 사전 훈련된 네트워크의 합성곱층(가중치 고정)에 새로운 데이터를 통과시킨 후, 그 출력을 데이터 분류기에서 훈련시킴.

- 합성곱층은 가중치 고정

+ 여기서 사용 가능한 이미지 분류 모델 : Xception, Inception V3, ResNet50, VGG16, VGG19, MobileNet



```
data_path = './train'
```

```
transform = transforms.Compose(  
    [  
        transforms.Resize([256, 256]),  
        transforms.RandomResizedCrop(224),
```

```

        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
    ])
train_dataset = torchvision.datasets.ImageFolder(
    data_path,
    transform=transform
)
train_loader = torch.utils.data.DataLoader(
    train_dataset,
    batch_size=32,
    num_workers=8,
    shuffle=True
)

print(len(train_dataset))

```

1 torchvision.transform은 이미지 데이터를 변환하여 모델의 입력으로 사용할 수 있게 변환 해주는 역할

- **파라미터 정의**

1. **Resize** : 이미지 크기를 조정. 즉 ,256 X 256크기로 이미지 데이터를 조정한다.
2. **RandomResizedCrop** : 이미지를 랜덤한 크기 및 비율로 자르는 방법.
Resize 가 합성곱층을 통과하기 위해 이미지 크기를 조정하는 전처리 과정이라면,
RandomResizedCrop 은 데이터 확장 용도로 사용된다.
 이미지를 랜덤한 비율로 자른 후 데이터 크기를 조정하는 방법.
3. **RandomHorizontalFlip** : 이미지를 랜덤하게 수평으로 뒤집는다.
4. **ToTensor** : 이미지 데이터를 텐서로 변환한다.

2 datasets.ImageFolder 는 데이터로더가 데이터를 불러올 대상과 방법을 정의한다.

- **파라미터 정의**

1. 첫 번째 파라미터 : 불러올 데이터가 위치한 경로
2. **transform** : 이미지 데이터에 대한 전처리

3 DataLoader는 데이터를 불러오는 부분으로 앞에서 정의한 ImageFolder를 DataLoader에 할당하는데, 이때 한 번에 불러올 데이터양을 결정하는 batch_size와 데이터를 무작위로 섞을 것인지 설정한다.

- **파라미터 정의**

1. 첫 번째 파라미터 : 데이터셋을 지정
2. `batch_size` : 한 번에 불러올 데이터 양을 결정하는 배치 크기를 설정
3. `num_workers` : 데이터를 불러올 때 하위 프로세스를 몇 개 사용할 지 설정하는데, 이때 너무 많은 하위 프로세스를 설정하게 되면 오류가 발생하거나 메모리 부족 현상이 발생할 수 있다.
4. `shuffle` : 데이터를 무작위로 섞을지를 지정

➡ 이후 동시에 적기 번거로워서 코드와 함께 ipynb 파일에 서술하였습니다.

ResNet18

ResNet18은 50개의 계층으로 구성된 합성곱 신경망입니다. ImageNet 데이터베이스의 100만 개가 넘는 영상을 이용하여 훈련된 신경망으로 전이 학습에 사용되도록 사전 훈련된 모델을 제공하고 있습니다. 하지만 ResNet18은 입력 제약이 매우 크고, 충분한 메모리 (RAM)가 없으면 학습 속도가 느릴 수 있는 단점이 있습니다. 자세한 내용은 6장에서 배웁니다.

훈련과 테스트 데이터 정확도 그래프

훈련과 테스트 데이터 모두 에포크가 진행될수록 정확도가 높아지면서 100%에 가까워지고 있음. 실제로 더 많은 에포크를 진행한다면 더욱더 100%에 수렴하는 것을 확인할 수 있음.

훈련 데이터의 오차 그래프 확인

에포크가 진행될수록 오차가 낮아지고 있기 때문에 학습이 잘 되었다고 할 수 있다.

5.3.2 미세 조정 기법(fine-tuning)

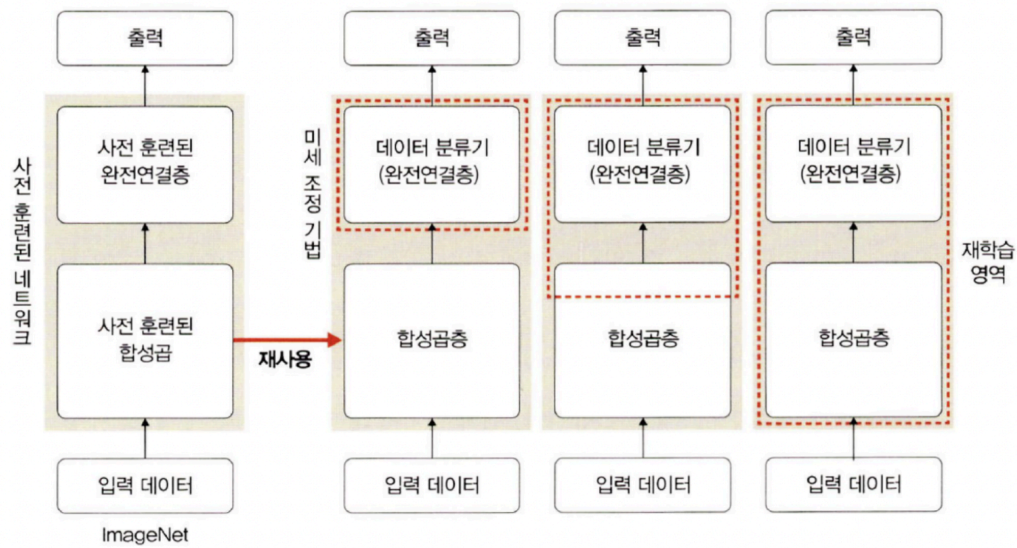
| 미세 조정 기법

특성 추출 기법에서 더 나아가 사전 훈련된 모델과 합성곱층, 데이터 분류기의 가중치를 업데이트하여 훈련시키는 방식. 특성 추출은 목표 특성을 잘 추출했다는 전제하에 좋은 성능을 낼 수 있다.

→ 특성이 잘못 추출되었다면 fine-tuning으로 새로운 이미지 데이터를 사용하여 네트워크의 가중치 업데이트하고 특성을 다시 추출할 수 있다.

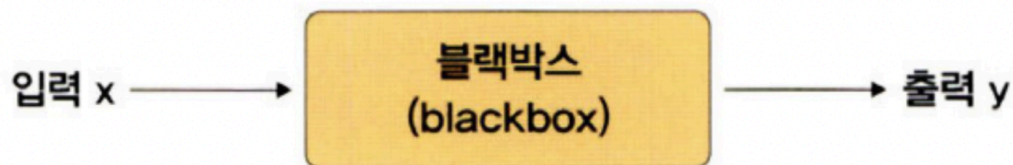
→ 즉, 사전 학습된 모델을 목적에 맞게 재학습시키거나 학습된 가중치의 일부를 재학습 시키는 것

- 파인 튜닝 과정에서 많은 연산량이 필요하기 때문에 CPU보다는 GPU 사용 권장
- 파인 튜닝 전략
 - 데이터셋이 크고 사전 훈련된 모델과 유사성이 작을 경우:
 - 모델 전체를 재학습시킨다. 데이터셋 크기가 크기 때문에 재학습이 좋은 전략.
 - 데이터셋이 크고 사전 훈련된 모델과 유사성이 클 경우:
 - 합성곱층의 뒷부분(완전연결층과 가까운 부분)과 데이터 분류기를 학습
 - 데이터셋이 유사하기 때문에 전체를 학습 시키는 것보다는 강한 특징이 나타나는 합성곱층의 뒷부분과 데이터 분류기만 새로 학습하더라도 최적의 성능을 낼 수 있다.
 - 데이터셋이 작고 사전 훈련된 모델과 유사성이 작을 경우:
 - 합성곱층의 일부분과 데이터 분류기를 학습
 - 데이터가 적기 때문에 일부 계층에 미세 조정 기법을 적용한다고 해도 효과가 없을 수 있다. 따라서 합성곱층 중 어디까지 새로 학습시켜야 할지 적당히 설정해 주어야 한다.
 - 데이터셋이 작고 사전 훈련된 모델과 유사성이 클 경우:
 - 데이터 분류기만 학습
 - 데이터가 적기 때문에 많은 계층에 미세 조정 기법을 적용하면 과적합이 발생할 수 있다.
 - 최종 데이터 분류기인 완전연결층에 대해서만 파인튜닝 적용.



5.4 설명 가능한 CNN

- 설명 가능한 CNN이란 딥러닝 처리 결과를 사람이 이해할 수 있는 방식으로 제시하는 기술이다.
- **CNN은 본질적으로 블랙박스에 가까워** 내부 표현(필터·활성맵·특징 조합)이 복잡해, 특정 예측이 왜 나왔는지 설명하기 어렵다.
- 이를 해결하기 위해 CNN 처리과정을 시각화할 필요성이 있음.



- 신뢰성을 위해선?
 - CNN을 구성하는 각 중간 계층부터 최종 분류까지 입력된 이미지에서 **특성이 어떻게 추출되고 학습하는지를 시각적으로 설명할 수 있어야 함.**
- 시각화 방법
 - 필터에 의한 시각화
 - 특성 맵에 대한 시각화

5.4.1 특성 맵 시각화

- 특성 맵(feature map)이란 입력 이미지 또는 다른 특성 맵처럼 필터를 입력에 적용한 결과
- 특성 맵 시각화 → 숫자 값으로 이뤄진 특성맵 값을 색상, 명암, 히트맵 등으로 변환해, "CNN이 이미지의 어떤 부분을 주목했는지" 사람이 볼 수 있게 보여주는 것.

➡ 이후 동시에 적기 번거로워서 코드와 함께 ipynb 파일에 서술하였습니다.

5.5 그래프 합성곱 네트워크

그래프 합성곱 네트워크(Graph Convolutional Network, GCN)
: 그래프 데이터를 위한 신경망

5.5.1 그래프란

- 그래프는 방향이 있거나 없는 엣지로 연결된 노드의 집합
- 구성
 - 노드(node, vertex): 보통 원으로 표시함
 - 엣지(edge): 두 노드를 연결한 선
- 노드 = 원소, 엣지 = 결합 방법 의미

5.5.2 그래프 신경망

그래프 신경망(Graph Neural Network, GNN)

- 그래프 구조에서 사용하는 신경망
- 그래프 데이터에 대한 표현 단계

1. 인접 행렬 (Adjacency Matrix, A)

- 정의: 노드 간의 연결 관계를 나타내는 $n \times n$ 크기의 행렬
- 규칙:
 - 노드 i와 j가 연결되면 $A[i][j] = 1$
 - 연결되지 않았으면 $A[i][j] = 0$
- 역할: **그래프의 구조 자체**를 수학적으로 표현

2. 특성 행렬 (Feature Matrix, X)

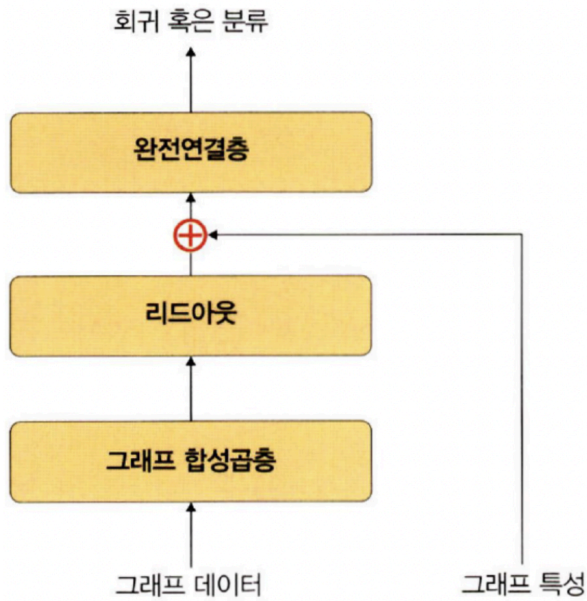
- 정의: 각 노드의 속성(특징)을 담고 있는 행렬
 - 각 행은 선택된 특성에 대해 각 노드가 갖는 값을 의미함.
 - 예) 첫 번째 행은 첫 번째 노드의 특성 값
- 규칙:
 - 행(Row) = 노드
 - 열(Column) = 해당 노드의 속성
- 특성 행렬 과정을 거쳐 그래프 특성이 추출됨.

5.3.3 그래프 합성곱 네트워크

| 그래프 합성곱 네트워크(Graph Convolutional Network, GCN)

이미지에 대한 합성곱을 그래프 데이터로 확장한 알고리즘.

- **그래프 합성곱 네트워크 구조**



- 리드아웃 층(readout): 특성 행렬을 하나의 벡터로 변환하는 함수.
 - 즉, 전체 노드의 특성 벡터에 대해 평균을 구하고 그래프 전체를 표현하는 하나의 벡터 생성.
- GCN에서 가장 중요한 부분 → 그래프 합성곱층
 - 그래프 합성곱층을 이용한 그래프 형태의 데이터는 **행렬 형태의 데이터로 변환되어** 딥러닝 알고리즘을 적용할 수 있기 때문
 - 기존 그래프 형태 데이터로는 딥러닝 알고리즘 적용 불가.
- GCN 활용 예시
 - SNS에서 관계 네트워크
 - 학술 연구에서 인용 네트워크
 - 3D Mesh